From Processes to Agents: Textual Models for Converting Process Logs into Agentic Systems

Marco Sánchez Sorondo[1,2] Claudio E. Righetti[1] Paulo Veiga[1]

[1]Departamento de Computación, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires, Argentina
[2]Universidad Austral, Argentina

RISE AI Summit 2026 · Santiago, Chile

The automation paradox

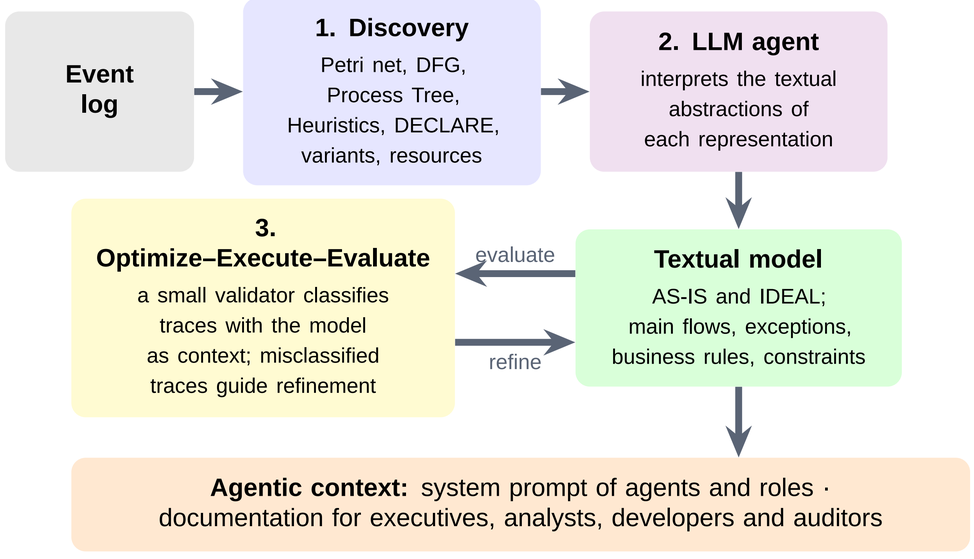
Agentic systems based on Large Language Models (LLMs) promise productivity gains, but designing, implementing, evaluating and maintaining them requires specialists in both business logic and agentic software. Their deployment often introduces processes more costly and complex than the workflows they replace.

- **Knowledge translation:** processes encode tacit rules, exceptions and heuristics that nobody wrote down.
- **Expertise scarcity:** Gartner predicts that over 40% of agentic AI projects will fail before 2027 [1].
- **Maintenance overhead:** agents need continuous monitoring and prompt refinement by the same scarce experts.

Meanwhile, **Process Mining** already turns event logs into formal models of how processes are actually executed, including exceptions and infrequent paths.

Agentic Process Mining (APM)

APM automatically generates a **textual process model** from event logs, designed as operational context for LLM-based agents and readable by people.



Discovery, generation and optimization are automatic. The optimized textual model is simultaneously the **process specification** for the analyst and the **operational context** for the agent.

A fidelity benchmark with a small validator

A small reasoning model (**GPT-5-mini**) receives the textual model and classifies each trace as VALID or INVALID, answering with one word, one trace per request. It works as a **clarity test**: process knowledge must reside in the textual model, not in the evaluator's parametric memory.

valid	Apply	Screen	Interview	Offer
swap	Apply	Interview	Screen	Offer

Invalid traces are generated by swapping two events of a valid trace. Four event logs from Pohl et al. [4] (Hiring, Hospital, Lending, Renting); 100 valid and 100 invalid traces per dataset; reference baseline: random classifier (F1 = 0.50).

Results across generators

Generator + validator	Hiring	Hospital	Lending	Renting	Mean
GPT-5 + GPT-5-mini	0.901	0.995	0.980	0.939	**0.954**
Grok 4.1 + GPT-5-mini	0.787	0.985	1.000	0.976	0.937
Claude Sonnet 4.5 + GPT-5-mini	0.791	0.995	0.980	0.976	0.936
DeepSeek V3.2 + GPT-5-mini	0.833	0.985	0.922	0.952	0.923
GPT-5-mini + GPT-5-mini	0.775	0.985	0.990	0.935	0.921
GPT-5-mini + GPT-5-nano	0.885	0.980	0.957	0.923	0.936

Textual models tend to be **permissive**: perfect recall, with errors concentrated in false positives. With GPT-5 as generator, only 40 errors over 800 classifications. GPT-5-mini + GPT-5-nano reaches 98% of the best F1 at roughly 2% of the cost. Rebmann et al. [3] report that LLMs operating without explicit process context perform comparably to a random classifier (F1 ≈ 0.50) on trace-level anomaly detection.

Adaptive explanations

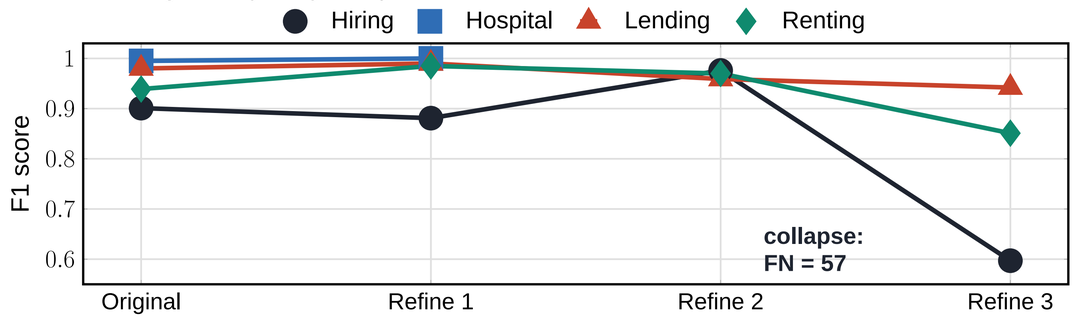
The optimized model was rewritten for two contrasting audiences and evaluated with the same benchmark.

Variant	F1	Recall	FP
Baseline	0.953	1.000	10.5
Executives	**0.969**	1.000	7.0
Children	**0.969**	1.000	6.5

Both adaptations outperform the baseline on average. In Hiring, the *children* version improves F1 from 0.873 to 0.943. Explaining the process in simple terms forces the adapter to remove ambiguous formulations.

Optimization and semantic overfitting

Misclassified traces act as **textual gradients** [2]: they guide the agent to refine the model, and the best model along the trajectory is kept.



	Hiring	Hospital	Lending	Renting
Original model F1	0.901	0.995	0.980	0.939
Best model F1	**0.976**	**1.000**	**0.990**	**0.985**
Optimal iteration	Refine 2	Refine 1	Refine 1	Refine 1

Mean F1 rises from 0.954 to **0.988**, with 80% fewer false positives (40 → 8). Refinement is non-monotonic: each iteration adds constraints, and in Refine 3 Hiring rejects 57% of valid traces. We call this **semantic overfitting**: overfitting in the space of textual descriptions rather than in parameter space.

There is no universal number of refinements: both FP and FN must be monitored, and the best model selected per process.

Decomposition by role for multi-agent systems

An agent clusters individual resources into logical roles (e.g., Loan Officer 1–5) and writes one model per role: activities, decisions, and from whom it receives and to whom it hands over work.

	Resources	Roles	F1 full	F1 by roles	ΔF1
Hiring	8	6	0.769	0.766	−0.003
Hospital	7	2	0.990	0.947	−0.043
Lending	19	4	0.990	0.990	+0.000
Renting	15	5	0.976	0.935	−0.041
Mean			0.931	0.910	−0.021

Per-role models concatenated into a single document preserve **97.9%** of the full model's F1 on average. Each role model can serve directly as the **system prompt of an autonomous agent**, and the handover-of-work network provides the interaction protocol between agents.

Extended benchmark: nine perturbation types

Nine perturbation types instead of one swap; 100 valid and 50 invalid traces per type and dataset; perturbed traces matching an observed variant are discarded. Same representations and models (GPT-5 + GPT-5-mini, unoptimized model): macro F1 **0.982**, recall 1.000.

Perturbation	Detected	Perturbation	Detected
Random swap	0.980	Insert unknown activity	0.865
Dependency-aware swap	0.990	Insert misplaced activity	0.870
Two swaps	0.995	Duplicate single activity	0.880
Swap + insertion	0.990	Delete first or last event	0.770
Delete mandatory activity	0.950		

Fraction of invalid traces detected, mean over the four datasets.

Order violations are detected almost always. The hardest cases are insertions and the deletion of the first or last event, detected in 77–88% of the traces.

Conclusions

- APM turns event logs into a **textual process model**: a universal intermediate representation for executives, analysts, developers, auditors and agentic systems.
- With the textual model as context, small models without fine-tuning classify traces with high F1 (0.954 mean, 0.988 after optimization), whereas LLMs without explicit process context perform like a random classifier.
- A capacity-limited validator shows both that the model captures the process semantics and that it is clear enough for small models, enabling low-cost, on-premise validation.
- Optimization is non-monotonic: **semantic overfitting** requires monitoring FP and FN and keeping the best model per process.
- Textual models adapt to other audiences preserving fidelity, and decompose by role into system prompts for multi-agent systems.
- The textual model is **part of the automation's code**: it should be versioned, reviewed and tested, with the benchmark as test suite.
- Limitations: simulated logs with descriptive activity names; binary classification; a single validator model. Decisions that affect people require human supervision.

[1] Gartner, *Over 40% of agentic AI projects will be canceled by end of 2027*, 2025. [2] M. Yuksekgonul et al., TextGrad, NeurIPS 2024. [3] A. Rebmann et al., Evaluating the ability of LLMs to solve semantics-aware process mining tasks, arXiv:2407.02310, 2024. [4] T. Pohl et al., A collection of simulated event logs for fairness assessment in process mining, arXiv:2306.11453, 2023.

Code